

MLS3

ENSC 254 Final Project - Report

Scope: This section documents the Milestone 3 implementation, validation, cycle accounting, cache-performance results, and cache-configuration exploration. The implementation extends the Milestone 2 five-stage pipeline simulator by adding a one-level data cache to the MEM stage.

1. Implementation

The cache simulator from Lab 4 was integrated into the final project through `cache.c` and the memory stage in `pipeline.c`. The main implementation changes were:

- **cache.c:** Implemented address-to-block conversion, tag and set extraction, cache probing, hit updates, insertion into invalid lines, LRU/LFU victim selection, replacement, allocation/deallocation, access statistics, trace printing, and `processCacheOperation()`.
- **pipeline.c:** Added `mem_access_counter` and called `processCacheOperation()` whenever a load or store reaches the MEM stage. The returned latency is added to `total_cycle_counter` after subtracting the one cycle already counted by `cycle_pipeline()`.
- **pipeline.h:** Declared `mem_access_counter` as an external global counter.
- **config.h:** Added the Milestone 3 compile-time configurations for detailed cache traces, cache summary testing, and no-cache testing.
- **DSE files:** Added `dse_1k.txt`, `dse_2k.txt`, `dse_4k.txt`, and `dse_8k.txt` in the exact required four-line format.

Latency handling:

A cache hit has a total latency of 2 cycles. A cache miss or eviction has a total latency of 102 cycles, consisting of the 2-cycle hit lookup plus the 100-cycle memory penalty. With the cache disabled, every data-memory access has a latency of 100 cycles. Since `cycle_pipeline()` already increments the cycle counter once, the MEM stage adds only 1, 101, or 99 extra cycles, respectively.

2. Functional Validation

The project was compiled with `make clean` followed by a `make` before each configuration. All three supplied Milestone 3 test modes completed successfully. A test was considered successful when `diff` produced no output, meaning the generated trace matched the supplied reference trace exactly.

Test mode	Programs tested	Result
cache_complete	LS, multiply, random, testset_1	All four traces matched
cache_summary	vec_xprod with cache	Trace matched
no_cache	vec_xprod without cache	Trace matched

3. Cycle Accounting for Test Category 1

For a cache-enabled run, the memory-stall total is calculated ($= \text{misses}(100) + (\text{hits} + \text{misses})$)

Equivalently, each hit contributes 1 additional cycle and each miss or eviction contributes 101 additional cycles beyond the normal pipeline cycle. The total cycle count is the base pipeline execution cycles plus the calculated memory stalls.

Test	Accesses	Hits	Misses	Manual MEM-stall calculation	MEM stalls	Base cycles	Total cycles
LS	16	7	9	$7(1) + 9(101) = 916$	916	$943 - 916 = 27$	943
multiply	2	1	1	$1(1) + 1(101) = 102$	102	$247 - 102 = 145$	247
random	4	3	1	$3(1) + 1(101) = 104$	104	$131 - 104 = 27$	131
testset_1	2	1	1	$1(1) + 1(101) = 102$	102	$248 - 102 = 146$	248

Example interpretation. In the LS test, 7 accesses hit and 9 accesses missed. The cache therefore adds $7 \times 1 + 9 \times 101 = 916$ cycles. The complete run requires 943 cycles, so the remaining 27 cycles come from normal pipeline execution and pipeline fill/drain behavior. The other three tests follow the same accounting method.

4. Performance Analysis of vec_xprod

Metric	With cache	Without cache
Total cycles	3,769,724	26,935,748
MEM stalls	1,493,688	24,659,712
Cache accesses	249,088	0
Cache hits	236,642	0
Cache misses	12,446	0

Hit rate: $236,642 / 249,088 \times 100 = 95.00\%$

Speedup: $26,935,748 / 3,769,724 = 7.15x$

The cache substantially improves performance. Without a cache, each load and store pays the full 100-cycle memory latency. With the cache, approximately 95% of the accesses hit and require only 2 total cycles. This reduces memory stalls by 23,166,024 cycles and lowers the overall execution time by approximately 86.0%.

5. Cache Configuration Exploration

The selected configurations are fully associative LRU caches with eight lines. Cache capacity is determined by $2^s \times E \times 2^b$ bytes. For all four configurations, $s = 0$ and $E = 8$; the block size increases with the target capacity.

File	Set bits	Associativity	Policy	Block bits	Capacity check
dse_1k.txt	0	8	LRU	7	$1 \times 8 \times 128 \text{ B} = 1 \text{ KB}$
dse_2k.txt	0	8	LRU	8	$1 \times 8 \times 256 \text{ B} = 2 \text{ KB}$
dse_4k.txt	0	8	LRU	9	$1 \times 8 \times 512 \text{ B} = 4 \text{ KB}$
dse_8k.txt	0	8	LRU	10	$1 \times 8 \times 1024 \text{ B} = 8 \text{ KB}$

Reason for the selected organization. The `vec_xprod` program accesses three-component vectors stored contiguously in memory. For each i iteration, `uvw[i][0-2]` and `xyz[i][0-2]` are loaded, while the inner j loop repeatedly traverses a 16-element tile of `xyz`. Large blocks exploit this strong spatial locality by bringing many adjacent vector elements into the cache with one miss. A fully associative organization removes set-index conflicts between the `uvw`, `xyz`, and result regions, whose base addresses are regularly spaced. LRU is appropriate because recently used blocks from the active 16-element tile are the blocks most likely to be reused before older blocks.

Effect of increasing capacity. The block bits increase from 7 to 10, so one fetched block covers progressively more consecutive vector data. This reduces compulsory misses during sequential and tiled access. The fully associative layout also allows the limited number of large blocks to be used by any address without conflict of misses caused by a fixed set mapping.

6. Conclusion

Milestone 3 was completed by integrating the Lab 4 cache model into the MEM stage of the five-stage pipeline, applying the specified cache and main-memory latencies, recording cache statistics, and supporting both cache-enabled and cache-disabled execution. All supplied reference tests have been passed. For `vec_xprod`, the cache achieved a 95.00% hit rate and a 7.15x speedup over the no-cache baseline, demonstrating that the tiled access pattern provides strong temporal and spatial locality.

